



Recibe una cálida:

¡Bienvenida!

Te estábamos esperando 😊 +

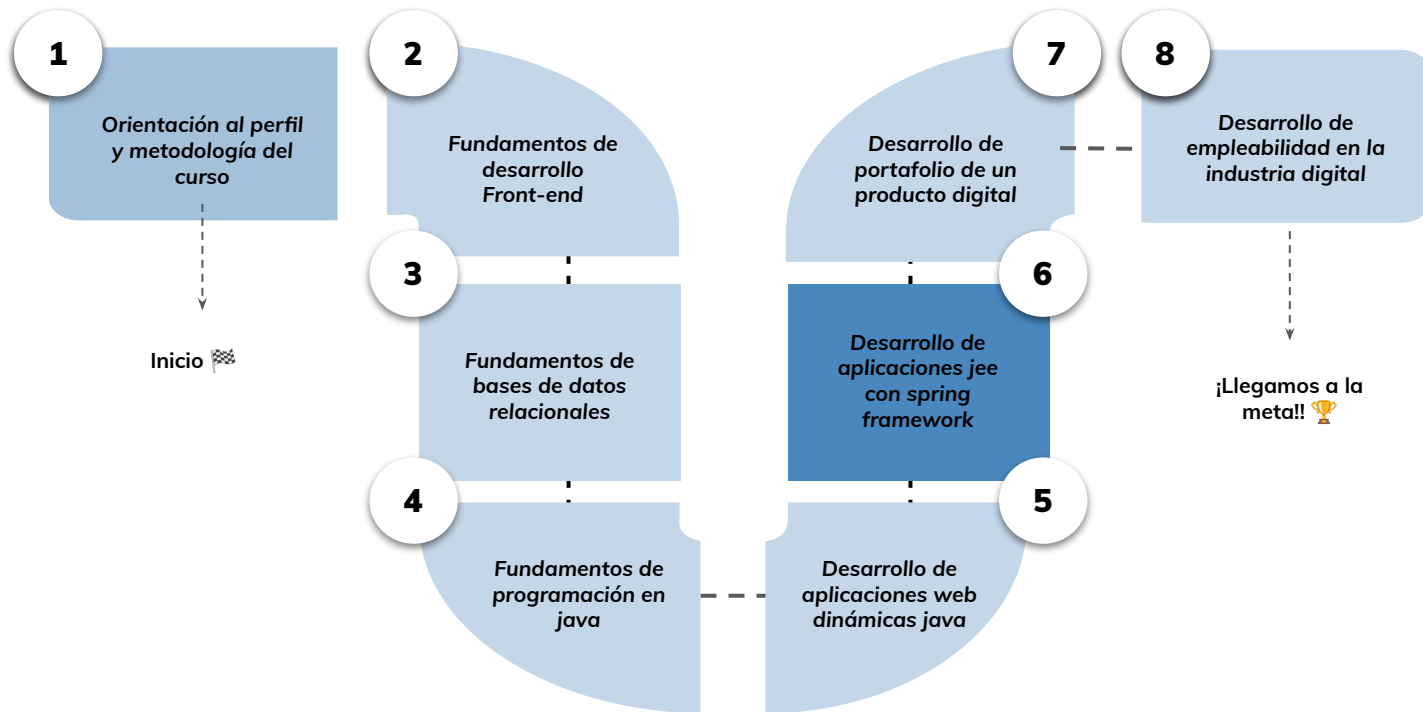


› La interoperabilidad entre los sistemas - Parte 2

Aprendizaje Esperado 5: Utilizar spring framework para la disponibilización de un servicio rest que da solución a un problema de interoperatividad.

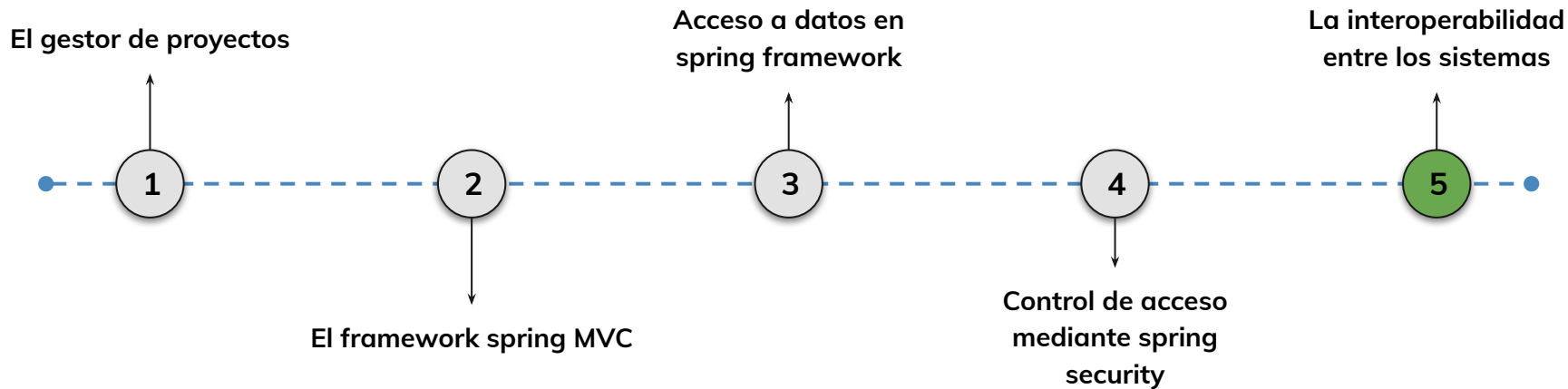
Hoja de ruta

¿Cuáles skills conforman el programa? **Desarrollo de Aplicaciones Full Stack Java Trainee**



Roadmap de lecciones

¿Cuáles **lecciones** estaremos estudiando en este módulo?



Learning Path

¿Cuáles temas trabajaremos hoy?

5.

Consumo y seguridad de servicios REST en Spring

Aprenderemos a consumir servicios externos mediante RestTemplate y aplicar mecanismos de autenticación y autorización con JWT en APIs REST construidas con Spring.

Consumo de servicios REST

Uso de RestTemplate

Buenas prácticas

Seguridad con JWT

Tokenización y filtros

Configuración en Spring Security

Objetivos de aprendizaje

¿Qué aprenderás?

- Comprender qué es RestTemplate y cómo usarlo.
- Aplicar métodos HTTP para consumir APIs externas.
- Implementar manejo de errores y validación de datos.
- Configurar autenticación con JWT en una API REST.
- Proteger endpoints y gestionar roles con Spring Security.

Repaso clase anterior

¿Quedó alguna duda?

En la clase anterior trabajamos :

- Definición y objetivos de la interoperabilidad entre sistemas
- Protocolos de intercambio de datos: HTTP, HTTPS, JSON, XML, REST y SOAP
- Principios RESTful aplicados a servicios web
- Implementación básica de un servicio REST en Spring usando JSON como formato de intercambio

➤ RestTemplate

< > RestTemplate

Spring Boot proporciona una forma conveniente de realizar solicitudes HTTP mediante el uso de la clase **RestTemplate** . Esta clase es una herramienta poderosa para realizar solicitudes a servicios web RESTful y se puede usar para solicitudes sincrónicas y asincrónicas.



< > RestTemplate

Un servicio **RESTful** expone recursos a través de **URIs**. Cada recurso puede tener múltiples representaciones, siendo **JSON** y **XML** las más comunes. Las operaciones **HTTP** definen las acciones sobre los recursos: **GET** para recuperar, **POST** para crear, **PUT** para actualizar y **DELETE** para eliminar.

```
GET /usuarios/1 // Obtener información sobre el usuario con ID 1
POST /usuarios // Crear un nuevo usuario
PUT /usuarios/1 // Actualizar información del usuario con ID 1
DELETE /usuarios/1 // Eliminar al usuario con ID 1
```



< > RestTemplate

RestTemplate es una clase de Spring que simplifica la interacción con servicios RESTful. Proporciona métodos para realizar operaciones HTTP y gestionar la serialización y deserialización de datos en formato JSON o XML.

Para poder utilizarla es necesario tener la dependencia spring-web e importar la clase necesaria:

```
import org.springframework.web.client.RestTemplate;
```

```
@Autowired  
RestTemplate restTemplate;
```



< > RestTemplate

Mejores Prácticas para consumir servicios RESTful

- **Manejo de Excepciones** : Implementar manejo adecuado de excepciones para gestionar errores de red, problemas de autenticación y otros escenarios inesperados.
- **Paginación y Filtrado** : Utilizar paginación y filtrado en las solicitudes para optimizar el rendimiento y reducir el tamaño de las respuestas.
- **Caching** : Aplicar estrategias de caching para evitar solicitudes redundantes y mejorar la eficiencia.



< > RestTemplate

Consideraciones de Seguridad a la hora de consumir servicios RESTful

- **Autenticación y Autorización:** Implementar mecanismos de autenticación (como tokens) y autorización para proteger el acceso a los recursos.
- **Conexiones Seguras (HTTPS):** Utilizar conexiones seguras para proteger la confidencialidad y la integridad de los datos transmitidos.
- **Validación de Entradas:** Validar y sanitizar las entradas para prevenir ataques comunes como inyecciones SQL o XSS.



< > RestTemplate

Los **principales métodos** que provee **RestTemplate** en Spring **para consumir servicios** REST son:

- `getForObject()`
- `getForEntity()`
- `postForObject()`
- `postForEntity()`
- `put()``delete()`
- `exchange()`

Estos métodos permiten de forma sencilla :

- Realizar operaciones con los principales verbos HTTP (GET, POST, PUT, DELETE).
- Enviar objetos Java en el cuerpo y recibir objetos mapeados desde la respuesta.
- Obtener metadatos de la respuesta en `ResponseEntity`.
- Intercambiar datos en formatos como JSON.
- Manejar errores y excepciones de forma transparente.



< > RestTemplate

Realizar solicitudes GET

Para realizar una solicitud GET, puede utilizar los métodos `getForObject()` o `getForEntity()`.

El método `getForObject()` devuelve el cuerpo de la respuesta como un objeto, mientras que el método `getForEntity()` devuelve la respuesta completa, incluidos los encabezados y el código de estado.

```
// Realiza una solicitud GET y devuelve el cuerpo de la respuesta como un objeto Usuario
Usuario usuario = restTemplate.getForObject( "https://api.example.com/usuarios/{id}" , Usuario.class,1);

// Realiza una solicitud GET y devuelve la respuesta completa
ResponseEntity<Usuario> usuario2 = restTemplate.getForEntity("https://api.example.com/usuarios",Usuario.class);
```



< > RestTemplate

Realizar solicitudes POST

Para realizar una solicitud POST, puede utilizar los métodos `postForObject()` o `postForEntity()`.

Estos métodos funcionan de manera similar a los métodos GET, donde el método `postForObject()` devuelve el cuerpo de la respuesta como un objeto y el método `postForEntity()` devuelve la respuesta completa.

```
// Realiza una solicitud POST de creación de Usuario y devuelve el cuerpo de la respuesta como un objeto Usuario
Usuario usuario = restTemplate.postForObject("https://api.example.com/usuarios", new Usuario(), Usuario.class);

// Realiza una solicitud POST y devuelve la respuesta completa
ResponseEntity<Usuario> respuesta1 = restTemplate.postForEntity("https://api.example.com/usuarios", new Usuario(), Usuario.class);
```



< > RestTemplate

Realizar solicitudes PUT y DELETE

Para realizar solicitudes PUT y DELETE, puede utilizar los métodos `put()` y `delete()` respectivamente.

En este ejemplo, estamos realizando una solicitud **PUT** a la URL que representa el recurso del usuario con ID 1. Estamos enviando el objeto `usuarioActualizado` como parte del cuerpo de la solicitud, y `RestTemplate` se encarga de serializarlo automáticamente en el formato correcto (por ejemplo, JSON o XML) antes de enviar la solicitud al servidor.

```
RestTemplate restTemplate = new RestTemplate();
String url = "https://api.example.com/usuarios/{id}";
Usuario usuarioActualizado = new Usuario("UsuarioActualizado", 30);

restTemplate.put(url, usuarioActualizado, 1);
```



< > RestTemplate

Realizar solicitudes PUT y DELETE

La operación HTTP DELETE se utiliza para eliminar recursos en el servidor. RestTemplate ofrece el método delete para realizar solicitudes DELETE.

En este ejemplo, estamos realizando una solicitud **DELETE** a la URL que representa el recurso del usuario con ID 1. RestTemplate se encarga de construir y enviar la solicitud DELETE al servidor.

```
RestTemplate restTemplate = new RestTemplate();  
String url = "https://api.example.com/usuarios/{id}";  
  
restTemplate.delete(url, 1);
```



< > RestTemplate

Casos de uso:

El RestTemplate típicamente se inyecta y utiliza en los **controllers** de Spring MVC. De esta forma, encapsulamos las llamadas a servicios externos en los controladores, manteniendo la lógica del negocio separada. El RestTemplate nos resulta muy útil en aplicaciones que consumen múltiples APIs.

```
@RestController
@RequestMapping("/api")
public class MyController {

    @Autowired
    RestTemplate restTemplate;

    @GetMapping("/users")
```



< > RestTemplate

Con **getForObject** enviamos una petición GET y recibimos el cuerpo de la respuesta directamente mapeado a una instancia de la clase **User**.

En el controlador agregaremos el objeto modelo y la vista a devolver en un **ModelAndView**.

```
@RestController
@RequestMapping("/api")
public class MyController {
    @Autowired
    RestTemplate restTemplate;

    @GetMapping("/users")
    public ModelAndView getUsers() {

        // Llamada a API externa
        List<User> users = restTemplate.getForObject("https://api.example.com/users", List.class);

        // Agregar al modelo
        ModelAndView model = new ModelAndView("users");
        model.addObject("users", users);

        return model;
    }
}
```

< > RestTemplate

Luego, en la vista `users.jsp` accedemos al modelo

```
<%@ page contentType="text/html; charset=UTF-8" language="java" %>
<html>
<head>
  <title>Users</title>
</head>
<body>
  <h1>Users</h1>

  <ul>
    <c:forEach items="${users}" var="user">
      <li>${user.name} - ${user.email}</li>
    </c:forEach>
  </ul>

</body>
</html>
```



LIVE CODING

Controlador Rest: Vamos a configurar los controladores para que respondan al protocolo REST e implemente RestTemplate. Primero debemos anotar las clases controlador como @RestController

- ➔ Crear un controlador “buscarUsuario(id)” que implemente el método **getForEntity()** y devuelva la respuesta completa (ResponseEntity).

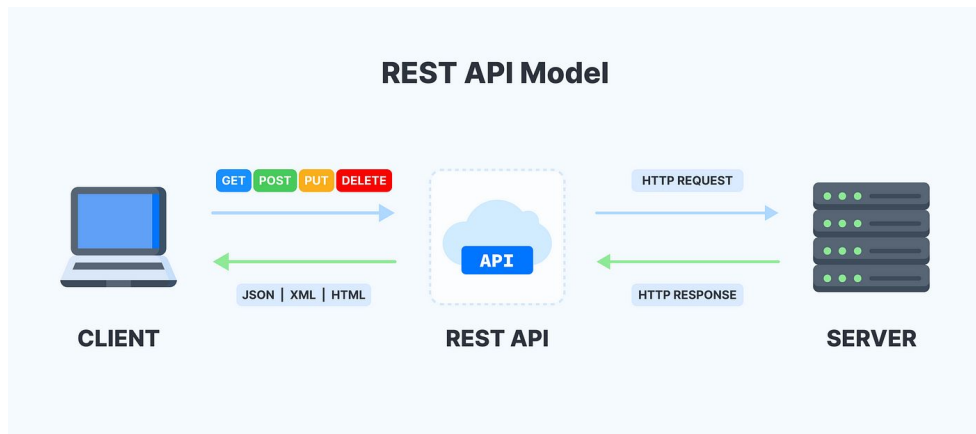
Tiempo: 15 minutos

➤ Principios de Diseño de una API REST



Principios de Diseño de una API REST

Una **API REST** es un conjunto de reglas y convenciones que **permite que dos aplicaciones se comuniquen entre sí a través de HTTP** de manera eficiente y escalable. Se basa en los principios de la arquitectura REST, que se centra en la simplicidad, la escalabilidad y la transferencia de representaciones de estado.



< > Principios de Diseño de una API REST

Identificación y Nomenclatura de Recursos : Uno de los principios fundamentales de una API REST es la identificación clara y consistente de recursos. Los recursos son entidades o conceptos que pueden ser accedidos o manipulados a través de la API. La nomenclatura de los recursos debe ser significativa y seguir convenciones para mejorar la comprensión y la usabilidad.

GET /usuarios : Obtener la lista de usuarios
GET /usuarios/{id} : Obtener información sobre un usuario específico
POST /usuarios : Crear un nuevo usuario
PUT /usuarios/{id} : Actualizar información de un usuario
DELETE /usuarios/{id} : Eliminar un usuario



< > Principios de Diseño de una API REST

Uso Apropiado de Métodos HTTP : Los métodos HTTP (GET, POST, PUT, DELETE, etc.) deben ser utilizados de acuerdo con su semántica específica. Por ejemplo, GET se utiliza para obtener información, POST para crear recursos, PUT para actualizar recursos y DELETE para eliminar recursos. Esta alineación con la semántica HTTP mejora la comprensión y previsibilidad de la API.



< > Principios de Diseño de una API REST

Uso de Formatos de Datos Estandarizados : El uso de formatos de datos estandarizados, como JSON o XML, facilita la interoperabilidad y el entendimiento. JSON ha ganado popularidad debido a su simplicidad y ligereza.

Documentación Clara y Completa : Una documentación clara y completa es esencial para que los desarrolladores comprendan y utilicen la API de manera efectiva. Herramientas como Swagger o OpenAPI pueden facilitar la generación de documentación a partir del código fuente.



< > Principios de Diseño de una API REST

Paginación y Filtrado para Listas de Recursos : Cuando se devuelven listas de recursos, se deben proporcionar mecanismos de paginación y filtrado para limitar la cantidad de datos transferidos y mejorar el rendimiento.

- `GET /usuarios?page=1&size=10` : Obtener la primera página de 10 usuarios
- `GET /usuarios?estado=activo` : Filtrar usuarios por estado activo



< > Principios de Diseño de una API REST

Respuestas HTTP Apropriadas y Códigos de Estado : Las respuestas HTTP deben proporcionar códigos de estado apropiados y significativos para indicar el resultado de la solicitud. Por ejemplo, **200 OK** para respuestas exitosas, **201 Created** para la creación de recursos, **204 No Content** para operaciones sin contenido, y códigos de error específicos para situaciones de error.

1XX	Códigos informativos	El servidor ha recibido la petición y procederá con ella.
2XX	Códigos de éxito	El servidor ha recibido, entendido y procesado la solicitud correctamente.
3XX	Códigos de redirección	El servidor ha recibido la solicitud, pero hay una redirección a alguna otra parte (o, en raras ocasiones, alguna acción adicional que debe completarse).
4XX	Códigos de error de cliente	El servidor no puede encontrar (o alcanzar) la página o la web. Se trata de un error del lado de la web.
5XX	Códigos de error de servidor	El cliente ha realizado una solicitud válida, pero el servidor ha fallado al completarla.

< > Principios de Diseño de una API REST

HATEOAS (Hypertext As The Engine Of Application State) : HATEOAS es un principio que sugiere que la aplicación del cliente debe ser guiada dinámicamente por enlaces incluidos en las respuestas de hipertexto de las aplicaciones servidoras. Esto facilita la navegación a través de la API sin la necesidad de entender previamente todas las posibles interacciones.

Webhooks y Event-Driven Architecture : La implementación de webhooks permite que la API notifique eventos a los clientes interesados en tiempo real. Esto se alinea con la arquitectura orientada a eventos, proporcionando una forma eficiente de manejar actualizaciones y cambios.



➤ Creando una API REST con Spring MVC

< > Creando una API REST con Spring MVC

Spring MVC (Model-View-Controller) es un **marco de trabajo** basado en el patrón de diseño Modelo-Vista-Controlador. Facilita la **creación de aplicaciones web y APIs REST** mediante la separación de responsabilidades en capas distintas. En particular, Spring MVC proporciona un enfoque robusto para la creación de servicios web RESTful.



< > Características de Spring MVC para APIs REST

1. Anotaciones para Mapeo de Solicitudes

Una de las características más destacadas de Spring MVC es su uso extensivo de **anotaciones** para mapear solicitudes HTTP a métodos específicos en controladores. Esto elimina la necesidad de configuraciones XML engorrosas y hace que el código sea más limpio y legible.



< > Características de Spring

MVC para APIs REST

En este ejemplo, la anotación `@RestController` indica que la clase es un controlador de Spring MVC que maneja solicitudes REST. La anotación `@RequestMapping` establece la ruta base para todas las operaciones definidas en la clase.

```
13 @RestController
14 @RequestMapping("/api/usuarios")
15 public class UsuarioControlador {
16
17     @GetMapping("/{id}")
18     public ResponseEntity<Usuario> obtenerUsuario(@PathVariable Long id) {
19         // Lógica para obtener y devolver un usuario por ID
20     }
21
22     @PostMapping
23     public ResponseEntity<String> crearUsuario(@RequestBody Usuario nuevoUsuario) {
24         // Lógica para crear un nuevo usuario
25     }
26 }
```



< > Características de Spring MVC para APIs REST

2. Conversión Automática de Datos a Formato JSON

Spring MVC facilita la creación de APIs que envían y reciben datos en formato JSON de manera transparente. Utiliza la biblioteca Jackson para la conversión automática de objetos Java a formato JSON y viceversa. Aquí, la respuesta del método `obtenerUsuario` se convertirá automáticamente a JSON antes de ser enviada al cliente.

```
@GetMapping("/{id}")  
public ResponseEntity<Usuario> obtenerUsuario(@PathVariable Long id) {  
    // Lógica para obtener y devolver un usuario por ID  
    return ResponseEntity.ok(usuario);  
}
```

< > Características de Spring MVC para APIs REST

3. Soporte para Validación de Datos

Spring MVC ofrece un sólido soporte integrado para la validación de datos. Al utilizar anotaciones de validación como `@Valid` y `@RequestBody`, los desarrolladores pueden validar automáticamente los datos recibidos en las solicitudes. En este ejemplo, la anotación `@Valid` indica que el objeto `nuevoUsuario` debe ser validado antes de que la lógica del controlador sea ejecutada.

```
@PostMapping
public ResponseEntity<String> crearUsuario(@Valid @RequestBody Usuario nuevoUsuario) {
    // Lógica para crear un nuevo usuario
    return ResponseEntity.ok("Usuario creado correctamente");
}
```



Ventajas de crear una API REST

Facilidad de Configuración y Desarrollo:

Una de las principales ventajas de Spring MVC es su facilidad de configuración. Al aprovechar las convenciones de opinión y la configuración basada en anotaciones, los desarrolladores pueden crear APIs de manera rápida y eficiente.

Con **Spring MVC**

Abstracción de Detalles de Bajo Nivel:

Spring MVC abstrae muchos de los detalles de bajo nivel relacionados con la manipulación de solicitudes HTTP y la serialización de datos. Esto permite a los desarrolladores centrarse en la lógica de negocio en lugar de preocuparse por la manipulación directa de la entrada y salida HTTP.

Ventajas de crear una API REST

Extensibilidad y Personalización:

La arquitectura de Spring MVC es altamente extensible. Los desarrolladores pueden personalizar y extender el comportamiento predeterminado mediante la implementación de interfaces y la configuración de componentes personalizados.

Con **Spring MVC**

Integración:

Spring MVC se integra de manera natural con otras características clave de Spring, como Spring Security para la implementación de seguridad, Spring Data para la integración con bases de datos, y Spring Boot para el desarrollo de aplicaciones autónomas.

Desventajas de crear una API REST

Curva de Aprendizaje Inicial:

Para los desarrolladores nuevos en Spring MVC, la curva de aprendizaje inicial puede ser empinada, especialmente si no están familiarizados con los conceptos fundamentales de Spring. Sin embargo, la abundancia de recursos en línea y la documentación oficial de Spring pueden mitigar este desafío.

Con **Spring MVC**

Complejidad en Proyectos a Gran Escala:

A medida que los proyectos crecen en complejidad y tamaño, la gestión de configuraciones y la comprensión completa de la arquitectura de Spring MVC pueden volverse desafiantes. Se recomienda un diseño cuidadoso y la implementación de mejores prácticas para abordar este desafío.



API A

API B

:8080/ejemploA/obtenerNombre

:8081/ejemploB/saludo



HOLA <NOMBRE
OBTENIDO>



< > Creando una API REST con Spring MVC

También permite hacer una llamada a una api externa y devolver una vista con la información a través del modelo, por ejemplo:

```
@GetMapping("/external")
public ModelAndView getExternalBooks() {

    String url = "https://api.example.com/books";
    List<Book> books = restTemplate.getForObject(url, List.class);

    ModelAndView mav = new ModelAndView("external");
    mav.addObject("books", books);

    return mav;
}
```



➤ Seguridad de una API REST mediante JWT

< > **Securización de una API REST mediante JWT**

Una de las formas más utilizadas actualmente para securizar APIs REST es mediante la implementación de **JSON Web Tokens (JWT)**. Los JWT permiten la autenticación stateless entre el cliente y el servidor, ya que encapsulan la información de acceso en un token firmado que se envía en cada request.



< > **Securización de una API REST mediante JWT**

Spring Framework provee el soporte necesario para incorporar JWT y seguridad basada en tokens dentro de una API REST creada con Spring MVC. Mediante la integración con Spring Security es posible agregar filtros de autenticación JWT, endpoints protegidos, roles y autorizaciones.



< > Seguridad de una API REST mediante JWT

Un JWT contiene tres partes :

1. **Header** : algoritmo usado (HS256, RS256, etc) y tipo de token.
2. **Payload** : claims o afirmaciones sobre la entidad. Puede contener datos como usuario, roles, etc.
3. **Firma** : permite verificar la integridad del mensaje.

El JWT se codifica en Base64 y se envía en el header Authorization de cada request.

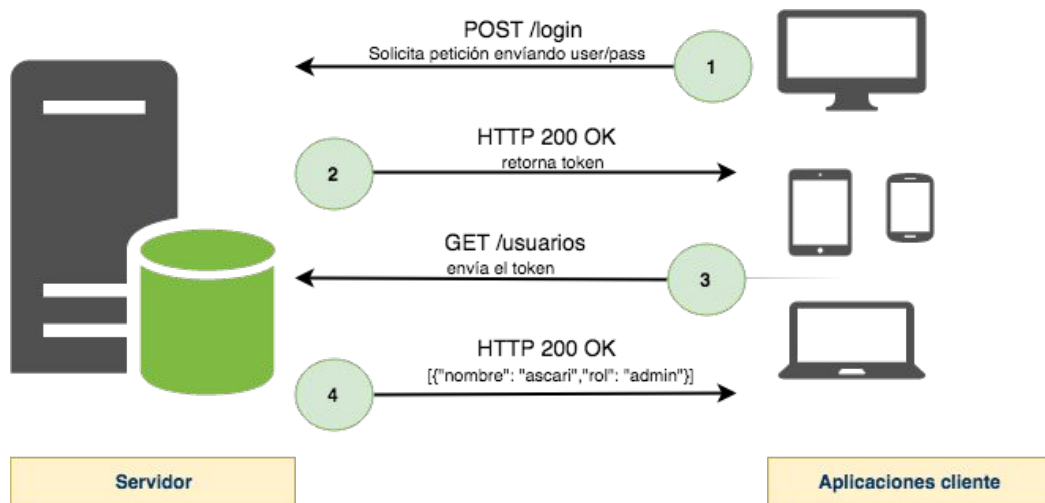
El servidor valida la firma para autenticar al cliente. Así se implementa la autenticación stateless.



< > **Securización de una API REST mediante JWT**

El siguiente diagrama muestra el flujo general de un proceso de autenticación basada en token.

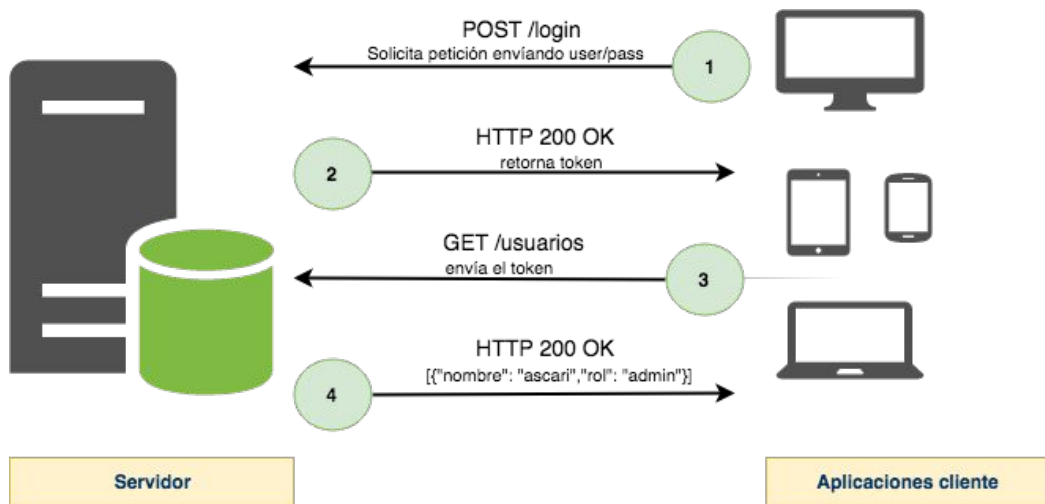
1. El cliente envía sus credenciales (usuario y password) al servidor.



< > Seguridad de una API REST mediante JWT

El siguiente diagrama muestra el flujo general de un proceso de autenticación basada en token.

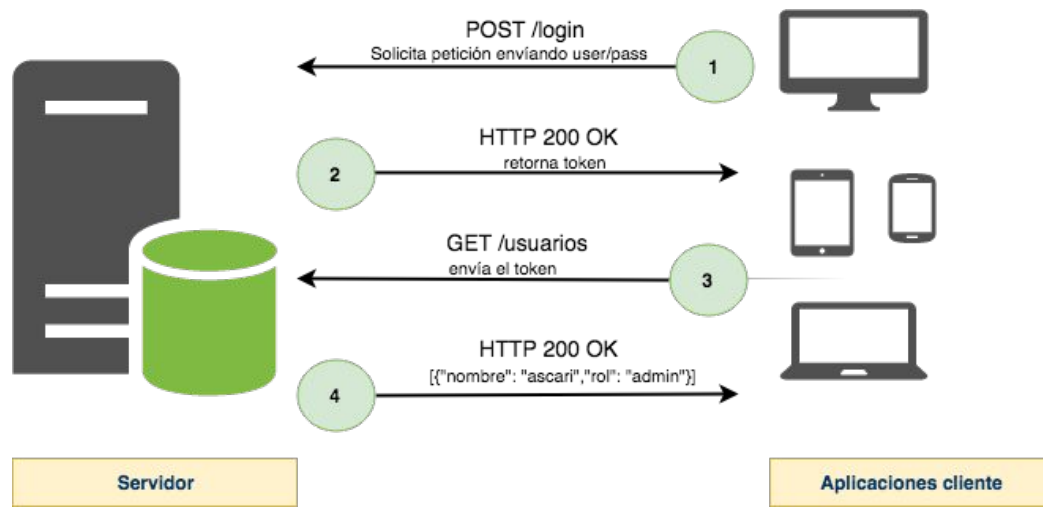
2. Si las credenciales son válidas, el servidor devuelve al cliente un token de acceso.



< > Seguridad de una API REST mediante JWT

El siguiente diagrama muestra el flujo general de un proceso de autenticación basada en token.

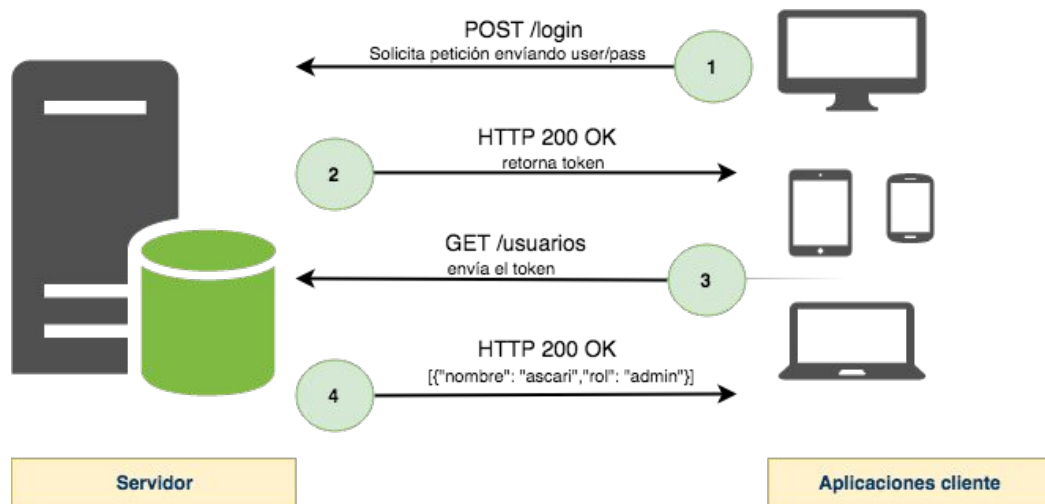
3. El cliente solicita un recurso protegido. En la petición, se envía el token de acceso.



< > Seguridad de una API REST mediante JWT

El siguiente diagrama muestra el flujo general de un proceso de autenticación basada en token.

4. El servidor valida el token y en caso de ser válido, devuelve el recurso solicitado.



< > **Securización de una API REST mediante JWT**

Veamos un ejemplo de un controlador REST para implementar el proceso de autenticación mediante un login usuario/contraseña securizado con JWT:

El método login(...) intercepta las peticiones POST al endpoint /user y recibirá como parámetros el usuario y contraseña.

```
@RestController
public class UserController {

    @PostMapping("/user")
    public User login(@RequestParam("user") String username, @RequestParam("password") String pwd) {

        String token = getJWTToken(username);
        User user = new User();
        user.setUser(username);
        user.setToken(token);
        return user;
    }
}
```



< > **Securización de una API REST mediante JWT**

Utilizamos el método **getJWTToken** (...) para construir el token, delegando en la clase de utilidad `Jwts` que incluye información sobre su expiración y un objeto de `GrantedAuthority` de Spring que, como veremos más adelante, usaremos para autorizar las peticiones a los recursos protegidos.

```
private String getJWTToken(String username) {  
    String secretKey = "mySecretKey";  
    List<GrantedAuthority> grantedAuthorities = AuthorityUtils  
        .commaSeparatedStringToAuthorityList("ROLE_USER");  
  
    String token = Jwts  
        .builder()  
        .setId("softtekJWT")  
        .setSubject(username)  
        .claim("authorities",  
            grantedAuthorities.stream()  
                .map(GrantedAuthority::getAuthority)  
                .collect(Collectors.toList()))  
        .setIssuedAt(new Date(System.currentTimeMillis()))  
        .setExpiration(new Date(System.currentTimeMillis() + 600000))  
        .signWith(SignatureAlgorithm.HS512,  
            secretKey.getBytes()).compact();  
  
    return "Bearer " + token;  
}
```



< > **Securización de una API REST mediante JWT**

Por último, editaremos nuestra clase de configuración de seguridad para añadir la siguiente configuración:

```
@Override
protected void configure(HttpSecurity http) throws Exception {
    http.csrf().disable()
        .addFilterAfter(new JWTAuthorizationFilter(), UsernamePasswordAuthenticationFilter.class)
        .authorizeRequests()
        .antMatchers(HttpMethod.POST, "/user").permitAll()
        .anyRequest().authenticated();
}
```

En este caso se permiten todas las llamadas al controlador /user, pero el resto de las llamadas requieren autenticación.

En este momento, si reiniciamos la aplicación y hacemos una llamada a `http://localhost:8080/hello`, nos devolverá un error 403 informando al usuario de que no está autorizado para acceder a ese recurso que se encuentra protegido.



< > **Securización de una API REST mediante JWT**

Ahora necesitamos implementar el proceso de autorización, que sea capaz de interceptar las invocaciones a recursos protegidos para recuperar el token y determinar si el cliente tiene permisos o no. Para ello implementaremos el siguiente filtro, JWTAuthorizationFilter:

```
23 public class JWTAuthorizationFilter extends OncePerRequestFilter {  
24  
25     private final String HEADER = "Authorization";  
26     private final String PREFIX = "Bearer ";  
27     private final String SECRET = "mySecretKey";  
28 }
```



< > **Securización de una API REST mediante JWT**

Luego agregaremos los métodos: **doFilterInternal** y **validateToken**:

```
@Override
protected void doFilterInternal(HttpServletRequest request, HttpServletResponse response, FilterChain chain)
    try {
        if (existeJWTToken(request, response)) {
            Claims claims = validateToken(request);
            if (claims.get("authorities") != null) {
                setUpSpringAuthentication(claims);
            } else {
                SecurityContextHolder.clearContext();
            }
        } else {
            SecurityContextHolder.clearContext();
        }
        chain.doFilter(request, response);
    } catch (ExpiredJwtException | UnsupportedJwtException | MalformedJwtException e) {
        response.setStatus(HttpServletResponse.SC_FORBIDDEN);
        ((HttpServletResponse) response).sendError(HttpServletResponse.SC_FORBIDDEN, e.getMessage());
        return;
    }
}

private Claims validateToken(HttpServletRequest request) {
    String jwtToken = request.getHeader(HEADER).replace(PREFIX, "");
    return Jwts.parser().setSigningKey(SECRET.getBytes()).parseClaimsJws(jwtToken).getBody();
}
```

< > **Securización de una API REST mediante JWT**

En la misma clase agregaremos los métodos necesarios para otorgar permisos

```
/**
 * Metodo para autenticarnos dentro del flujo de Spring
 *
 * @param claims
 */
private void setUpSpringAuthentication(Claims claims) {
    @SuppressWarnings("unchecked")
    List<String> authorities = (List) claims.get("authorities");

    UsernamePasswordAuthenticationToken auth = new UsernamePasswordAuthenticationToken(claims.getSubject(), null,
        authorities.stream().map(SimpleGrantedAuthority::new).collect(Collectors.toList()));
    SecurityContextHolder.getContext().setAuthentication(auth);
}

private boolean existeJWTToken(HttpServletRequest request, HttpServletResponse res) {
    String authenticationHeader = request.getHeader(HEADER);
    if (authenticationHeader == null || !authenticationHeader.startsWith(PREFIX))
        return false;
    return true;
}
```



< > **Securización de una API REST mediante JWT**

Este filtro intercepta todas las invocaciones al servidor (extiende de `OncePerRequestFilter`) y:

1. Comprueba la existencia del token (`existeJWTToken(...)`).
2. Si existe, lo desencripta y valida (`validateToken(...)`).
3. Si está todo OK, añade la configuración necesaria al contexto de Spring para autorizar la petición (`setUpSpringAuthentication(...)`).

Para este último punto, se hace uso del objeto **GrantedAuthority** que se incluyó en el token durante el proceso de autenticación.





Momento:

Time-out!



5 -10 min.





Ejercicio N° 1

Consumir y proteger una API REST

Consumir y proteger una API REST

Contexto: 🙌

Estás desarrollando un sistema que debe conectarse con otra API para consultar información de usuarios. A su vez, el sistema debe proteger los endpoints internos con autenticación basada en tokens JWT.

Consigna: 📝

Implementa una API REST con dos responsabilidades:

- **GET externo:** Consumir un endpoint público usando RestTemplate.
- **JWT interno:** Agregar autenticación JWT para proteger un endpoint local.

Tiempo 🕒: 30 minutos

Consumir y proteger una API REST

Paso a paso:

1. Crear un nuevo proyecto con Spring Initializr con las dependencias: Web, DevTools, Spring Security.
2. Implementar una clase Usuario para mapear datos externos.
3. Crear un controlador que consuma `https://jsonplaceholder.typicode.com/users/{id}` usando `getForEntity`.
4. Configurar Spring Security y agregar un login que devuelva un JWT válido.
5. Crear un filtro JWT personalizado (`JWTAuthorizationFilter`) para verificar el token.
6. Proteger el endpoint `/usuarios` y permitir solo usuarios autenticados.
7. Probar consumo externo y protección con Hoppscotch.io.
8. Validar el flujo completo y analizar los headers HTTP.

¿Alguna consulta?



Resumen

¿Qué logramos en esta clase?

- ✓ Usamos RestTemplate para hacer peticiones externas.
- ✓ Comprendimos cómo usar getObject y getForEntity.
- ✓ Aplicamos buenas prácticas como validación, filtrado y manejo de errores.
- ✓ Entendimos cómo funcionan los tokens JWT y sus componentes.
- ✓ Protegimos endpoints mediante autenticación con JWT.
- ✓ Configuramos filtros de autorización personalizados.
- ✓ Probamos el flujo de autenticación y acceso seguro.
- ✓ Integramos consumo externo y seguridad en un mismo proyecto.



¡Ponte a prueba!

Momento de ejercitación

Te invitamos a aprovechar esta última sección del espacio sincrónico para realizar de manera individual las **actividades disponibles en la plataforma**. Estas propuestas son clave para afianzar lo trabajado y **forman parte obligatoria del recorrido de aprendizaje**.

👉 **Análisis de caso** ————— 👉 **Selección Múltiple**

👉 **Comprensión lectora**

Si al resolverlas surge alguna duda, compartela o tráela al próximo encuentro sincrónico.

< **¡Muchas gracias!** >

